

CSE 579 Homework 3

Hisham Bhatti

1 Code Overview

2 Random MPC with Shooting

1. Fill in the blanks in the code marked with TODO in the `rollout_model` function in `planning.py`

See Gradescope submission for code.

2. Fill in the blanks in the code marked with TODO in the `plan_and_get_ensemble_rewards_model_random_shooting` function in `planning.py`

See Gradescope submission for code.

3. Plot the reward during the training with default hyperparameters. Report the success rate and average reward using the `evaluate` function that is provided to you.



Success Rate: 0.32

Average Reward (success only): -20.3359

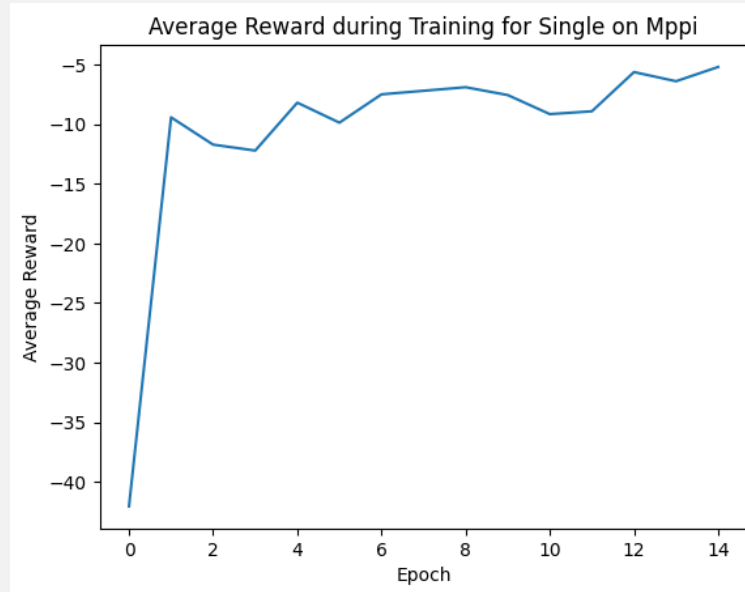
Average Reward (all): -22.2832

3 Model Predictive Path Integral (MPPI)

1. Fill in the blanks in the TODOs in the `plan_model_mppi` function in `planning.py`

See Gradescope submission for code.

2. Plot the reward during the training with default hyperparameters. Report the success rate and average reward using the `evaluate` function that is provided to you.



Success Rate: 0.56

Average Reward (success only): -5.8924

Average Reward (all): -7.2067

4 Ensemble MPPI

1. Implement the `get_ensemble_rewards` method in `planning.py`

See Gradescope submission for code.

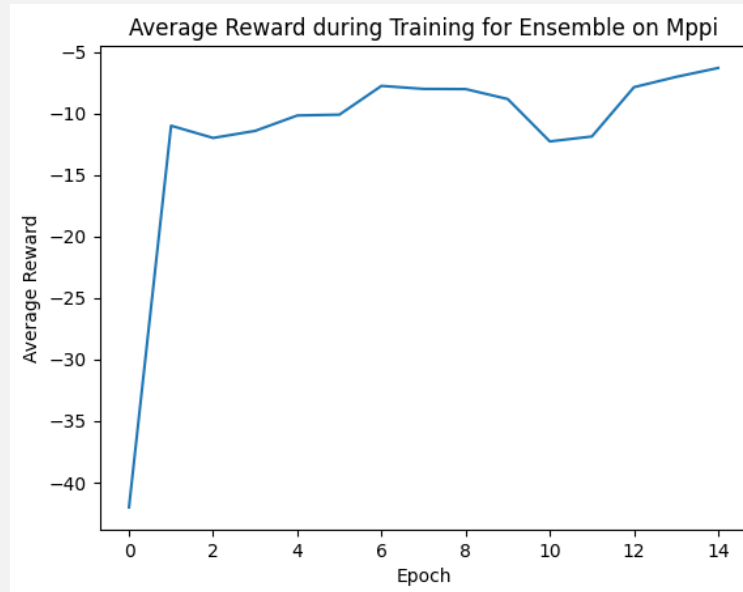
2. Add Ensemble MPPI in the `plan_model_mppi` function in `planning.py`. Your goal here is to iteratively rollout each model and average returns over models.

See Gradescope submission for code.

3. Fill in the blanks in the code marked with `TODO` in `train_model` function in `train_model.py`. Your goal here is to train each model with a different batch size.

See Gradescope submission for code.

4. Plot the return during the training with default hyperparameters. Report the success rate and average reward using the evaluate function that is provided for you.



Success Rate: 0.58

Average Reward (success only): -5.8430

Average Reward (all): -8.5000

5 Discussion

1. Compare Random MPC with shooting, MPPI and Ensemble MPPI. Discuss the strengths and weaknesses of each method.

All three of these model-based RL approaches decompose the problem into a model learning phase, where we construct a surrogate model $\hat{p}$ of the world through one-step supervised learning, similar to behavior cloning. The second part of the phase is planning, where we use this constructed model to pick actions along a trajectory. The naive random MPC with shooting simply performs random search in the planning phase, choosing random actions among many trajectories, estimating their rewards using the surrogate model, and choosing the first action which led to the highest reward. The trajectories are computed per step, with only using the first action from the optimal trajectory. This approach is simple, but in cases where the reward is sparse random shooting might not find an optimal trajectory. Thus, the approach is high variance.

Single MPPI improves on this technique via importance sampling: iteratively up-weighting the sampling distribution around trajectories which yield higher returns. We use the relative rewards along trajectories to inform our mean and standard deviation for sampling random actions. In doing so, MPPI achieves lower variance, and ultimately sticks closer to trajectories that yield high reward. However, just like in imitation learning with behavior cloning, both of these methods have the problem

of compounding error. This stems from a lack of data, where errors can push the model more OOD in a vicious cycle.

Ensemble MPPI aims to account for this by quantifying uncertainty. It trains several models (with diverse architectures, hyperparameters, etc.) and takes the average reward along the trajectories. While this helps with avoiding overly OOD settings, one weakness is a significant increase in computational time to train K models, which I noticed when completing this assignment.

2. Why is diversity important in an ensemble model? What techniques did you use in the homework to improve diversity in the ensemble MPPI?

A diversity of ensemble models is important because it helps capture uncertainty in low-data regimes, so that we can penalize such actions and guide our model. Ideally, we want our ensemble of models to agree on data within our domain so we encourage such actions, and have high variation OOD so actions that lead to those states are penalized. A diverse set of models ensure high disagreement in such regimes, allowing us to capture and penalize epistemic uncertainty.

In this homework, we adjusted the batch size and optimizer through learning rate. Modifying these hyperparameters ensured that the trained models will not perfectly match, especially in low-data regimes, which is necessary for the ensemble model. We did not change the underlying architecture across models; each were deep neural networks with the same number of layers and parameters.